

ENSF 380

TOPIC 19: INTERFACES

Interfaces

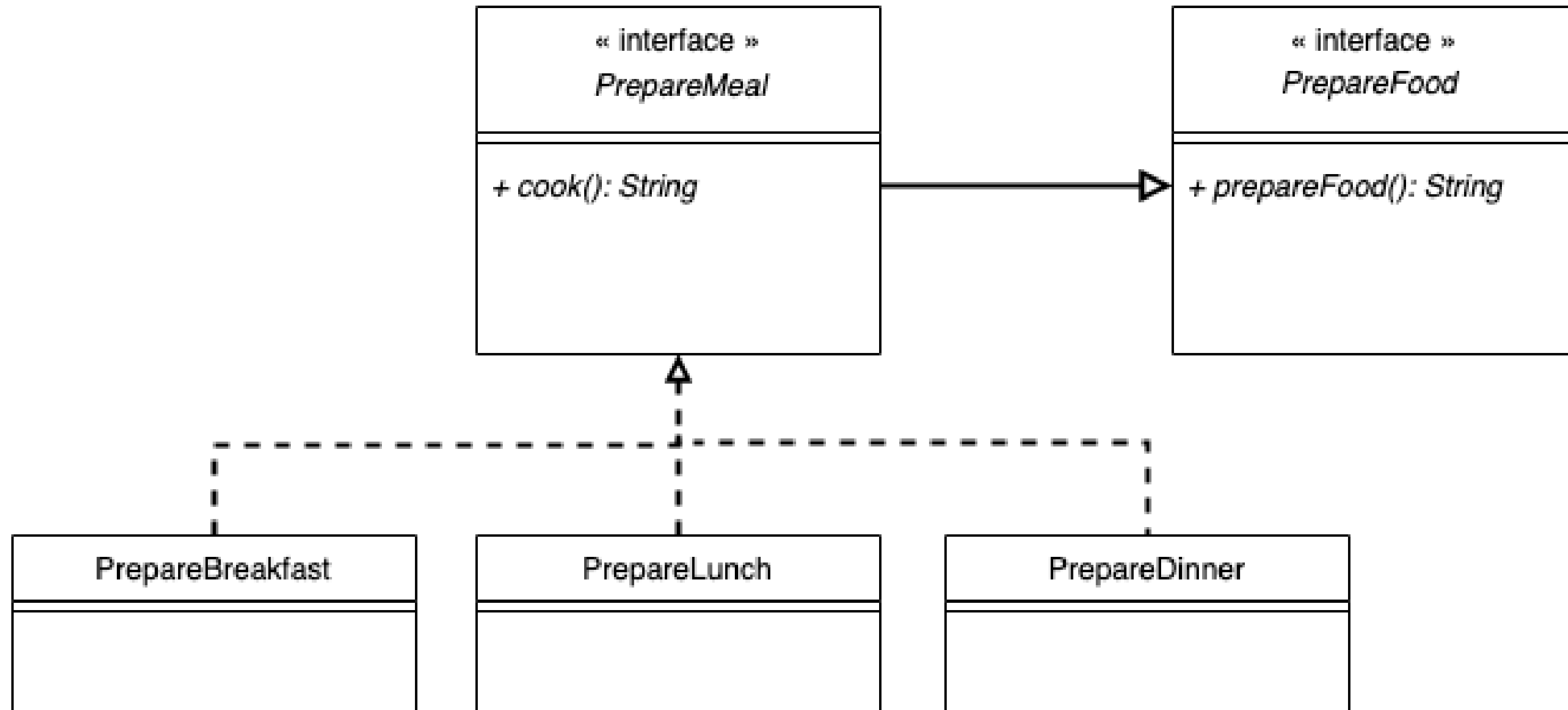
An interface declares a set of coherent public features and obligations

- Specifies a contract

Any class can implement an interface

- An instance of a class that realizes an interface must fulfil the contract
- Must provide code for all methods in the interface

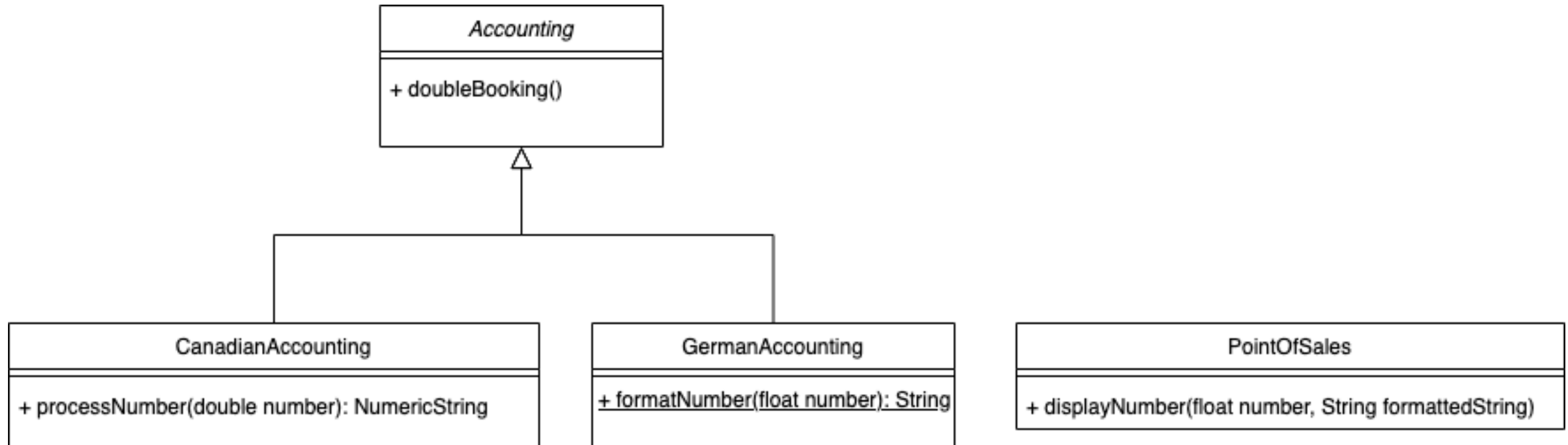
Interfaces in UML



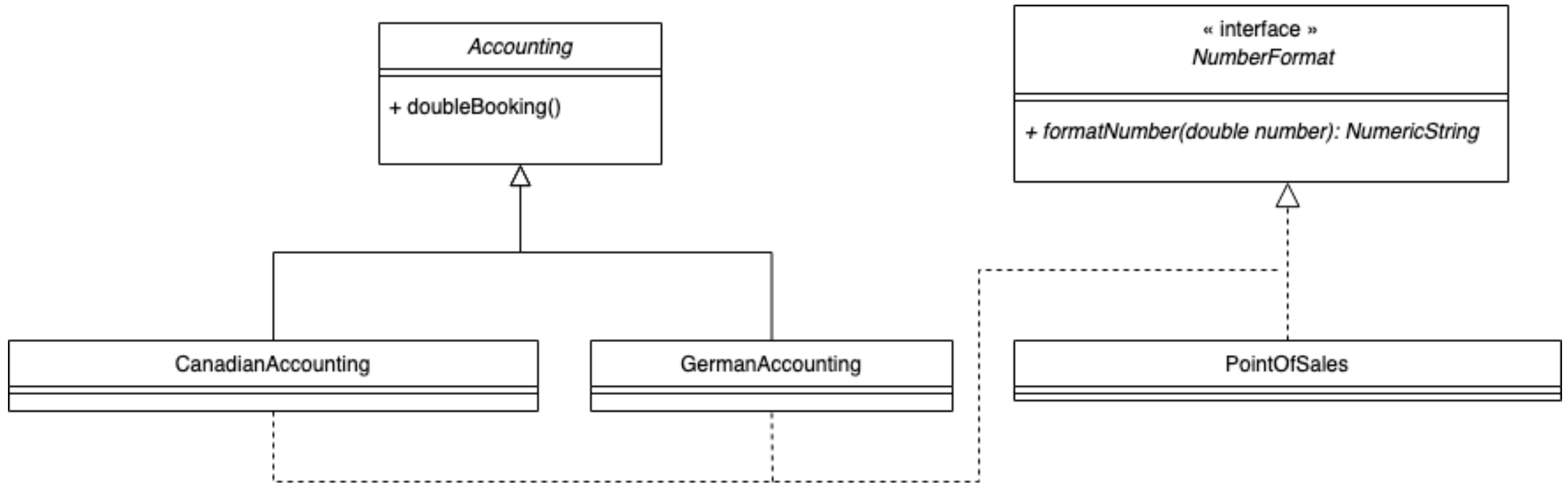
Differences Between Interfaces and Abstract Classes

Interface	Abstract Class
Is inherently abstract, without needing the abstract keyword	Is indicated with the abstract keyword
Uses a stereotype, italics in UML	Uses italics in UML
Is implemented (realized) by a class	Is extended by a class
Is extended by an interface	Cannot be extended by an interface
More than one interface can be implemented by a class	Only one (abstract or concrete) class can be extended by a class; a class can only have one parent
Cannot contain instance fields (all fields are public, static, and final)	Can contain instance fields; fields may have any visibility, fields may be final, static
Methods have public visibility	Concrete methods may have any visibility (abstract methods cannot be private)
Used when public behavior is important	Used when implementation details are important
Used for orthogonal hierarchies	Used for the primary hierarchy

Why Use Interfaces



Why Use Interfaces



Writing Interfaces

Interface

```
interface NumberFormat {  
    NumericString formatNumber(double number);  
}
```

Abstract class

```
abstract class NumberFormat {  
    public abstract NumericString formatNumber(double number);  
}
```

Realizing Interfaces

Lesson15/03_DeepCopy

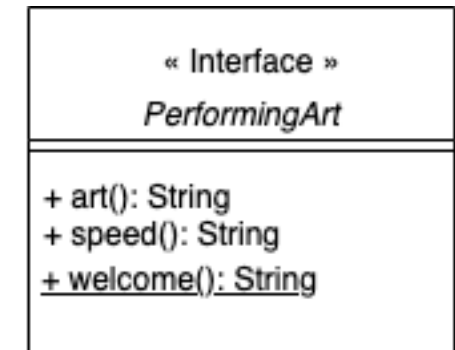
```
...  
class CompetitionHorse implements Cloneable{  
    private String animalSpecies;  
    private String animalFamily;  
    private String animalPhylum;  
    private String animalName;  
    private Rider animalRider = new Rider();  
    public String[] skills = new String[10];  
    ...
```

```
class Rider implements Cloneable{  
    private String riderName;  
    private int riderRegNumber;  
    private int age;  
    ...
```


Default Interface Methods

01_Song

```
interface PerformingArt {  
    default String art() {  
        return "Theater";  
    }  
  
    default String speed() {  
        return "Slow";  
    }  
  
    static String welcome() {  
        return "Welcome to the Performing Arts!";  
    }  
}
```



Extending an Interface

01_Song

```
interface Music extends PerformingArt {  
    // All fields are static and final  
    String INSTRUMENT = "Guitar";  
    String KEY = "Minor";  
  
    @Override  
    default String art() {  
        return "Music";  
    }  
  
    default String tempo() {  
        return "Adagio";  
    }  
}
```

from PerformingArt:

- art()
- speed()
- welcome()

added in Music

- art()
- tempo()
- INSTRUMENT
- KEY

Extending More Than One Interface

01_Song

```
interface Recording extends PerformingArt, SoundStudio {  
    default String art() {  
        return "Music!";  
    }  
  
    default double duration() {  
        return 5.06;  
    }  
  
    String producer();  
}
```

from PerformingArt:

- ~~art()~~
- speed()
- welcome()

from SoundStudio:

- studio()

added in Recording

- **art()**
- **duration()**
- ***producer()***

Inheriting and Implementing

01_Song

```
class Song extends Melody implements Music, Recording {  
  
    @Override  
    public String tune() {  
        return "dah-dah-dah-dum";  
    }  
  
    @Override  
    public String art() {  
        return Recording.super.art();  
    }  
  
    @Override  
    public String producer() {  
        return "University of Calgary productions";  
    }  
}
```

```
from Melody  
• tune()  
• tone()  
from Music  
• speed() [via P]  
• welcome() [via P]  
• art()  
• tempo()  
• INSTRUMENT  
• KEY  
from Recording  
• studio() [via S]  
• speed() [via P]  
• welcome() [via P]  
• art()  
• duration()  
• producer()
```

Exercise 19.1

TEST YOUR UNDERSTANDING BY COMPLETING THE EXERCISE

Invoke Super Class's Constructor (Inheritance)

```
super()
```

Invoke Method in Super Class (Inheritance)

```
super.methodName()
```

Invoke Interface's Default Method (Interfaces)

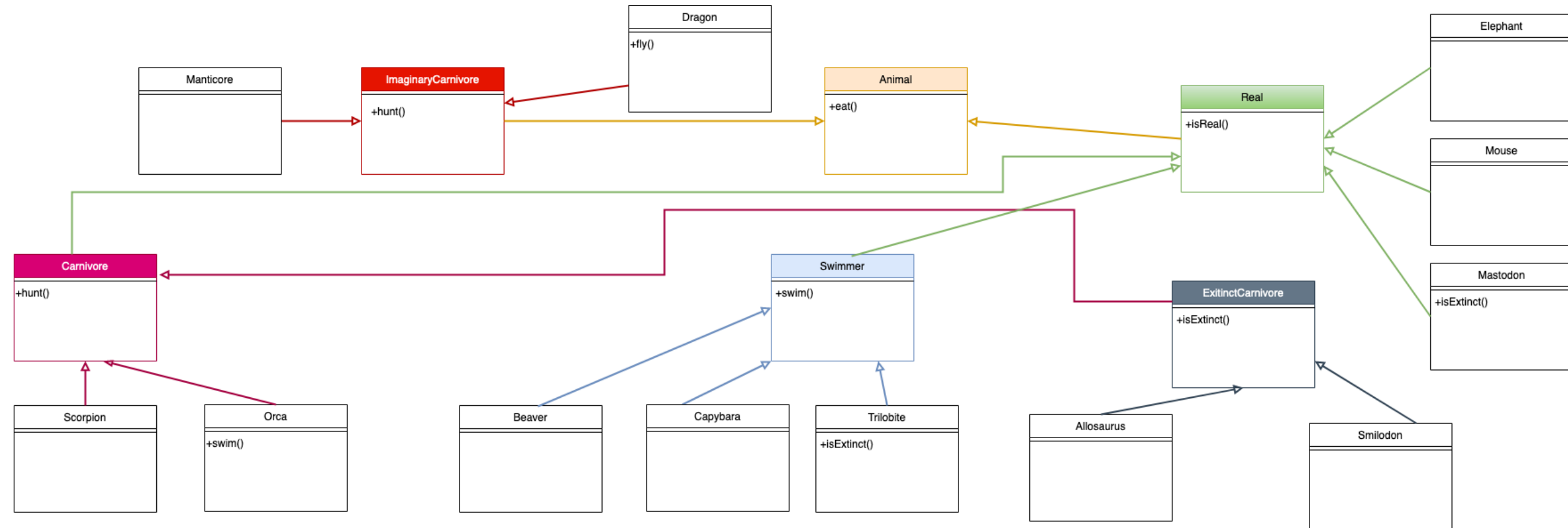
```
InterfaceName.super.methodName()
```

Implementing Interfaces

```
...  
public Object clone() throws CloneNotSupportedException{  
    CompetitionHorse copy = (CompetitionHorse)super.clone();  
    return copy;  
}  
...
```

Interfaces

Lesson18/04_Bestiary



Exercise 19.2

TEST YOUR UNDERSTANDING BY COMPLETING THE EXERCISE

Exercise 19.3

TEST YOUR UNDERSTANDING BY COMPLETING THE EXERCISE

Summary

Interfaces can extend (multiple) other interfaces

Classes can realize (multiple) interfaces

Interfaces are always abstract (keyword 'abstract' is not needed)

Interfaces are always public (keyword 'public' is not needed)

Methods in interfaces are always public (keyword 'public' is not needed)

Methods in interfaces may be abstract (keyword 'abstract' is not needed), static, or default.

- default as a method type should not be confused with default visibility, or as being the default setting for an interface method (abstract is the default).
- `InterfaceName.super.methodName()` is a way of accessing a default method in a specific interface
- `InterfaceName.methodName()` is a way of accessing a static method in a specific interface

Fields in interfaces are always public, static, and final (but don't require these keywords, and can be accessed via interface name)

- `InterfaceName.FIELD` is a way of accessing a field in an interface

Summary: Knowledge

Know what interfaces are

Know when to use interfaces and when to use abstract classes in Java

Understand the multiple uses of super in Java

Understand when you must override a method from an interface

Summary: Skills

Depict interfaces in UML

- Including fields and different types of methods

Write an interface in Java,

- With abstract, default, and static methods
- With fields

Extend an interface in Java

- Extend one interface
- Extend more than one interface

Realize an interface in Java

- Realize one interface
- Realize more than one interface
- Realize interfaces while inheriting
- Access interface methods through super

Additional Information

Related reading

- Volume 1, Chapter 6.1: Interfaces